MIT 8103 · ADVANCED DATABASE SYSTEMS · CA PORTFOLIO

PassPoint: Passport Appointment Slots, Portfolio Documentation

Student name	Matric number	Programme	Cohort
Moyinoluwa Emmanuel Idowu	2026/A/MIT/0077	Master of Information Technology	September 2026

GitHub repository: https://github.com/Emmanoel24/idowu-moyinoluwa-advanced-database-portfolio *(private; assessor added as collaborator)*

1. Selected case study

PassPoint is a simulated appointment-booking system (not a real government service) used by four passport offices in Lagos (Ikoyi, Alausa), Abuja and Port Harcourt. **Users:** applicants, front-desk and biometric officers, office supervisors and the system itself. **Data:** offices, counters, service types (fresh, renewal, replacement), 30-minute slots, applicants (national ID stored only as a SHA-256 hash), appointments, payments, status history and passport collections. **Operations:** book and reschedule a slot, pay, move an application through BOOKED → CONFIRMED → CAPTURED → PROCESSING → READY → COLLECTED, and report on demand, no-shows and turnaround. All data is invented; PostgreSQL 16 is the system of record and MongoDB 8 stores uploaded-document records.

2. Portfolio 1: Database design and modelling

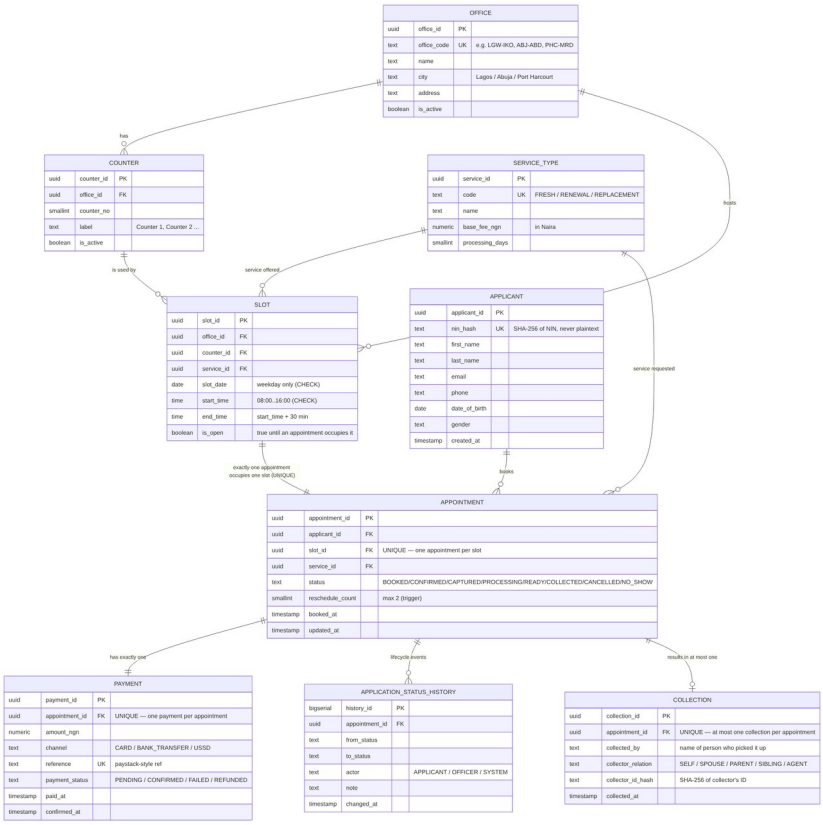


Figure 1. PassPoint ER diagram (9 entities).

The 9-table schema is in 3NF: office, counter and service details are stored once and referenced by keys. The database enforces the rules itself: one appointment per slot (UNIQUE), one active appointment per applicant per service (partial UNIQUE), weekday slots between 08:00 and 16:00 (CHECK), and triggers for payment-before-confirmation, a maximum of two reschedules and the status order. A reproducible generator loaded 30,000 applicants, 67,968 slots, 60,000 appointments and 60,000 payments; all 10 constraint tests were rejected with the intended error.

3. Portfolio 2: Query processing and optimisation

Query (EXPLAIN ANALYZE)	Before (ms)	After (ms)	Result
Q1 Free slots at an office on a date	3.15	0.10	~31× faster (index scan)
Q2 Appointments per office per month	84.12	73.22	small gain
Q4 Days from booking to collection	51.28	44.80	small gain
Q6 Revenue per service type	78.60	68.72	small gain

Query (EXPLAIN ANALYZE)	Before (ms)	After (ms)	Result
Q3 No-show rate · Q5 busiest times · Q7 revenue	46.8 · 48.8 · 153.3	51.7 · 57.4 · 156.0	no gain (Seq Scan kept)

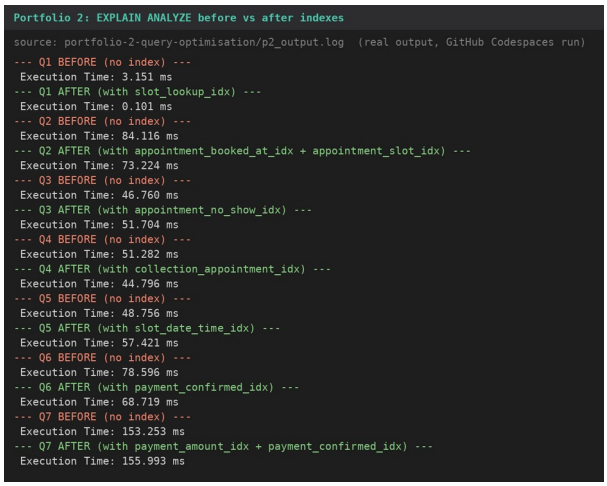


Figure 2. Real EXPLAIN ANALYZE times from p2_output.log.

The selective lookup (Q1) gained most, because the index reads a handful of rows. Reports that read most of a table kept a sequential scan, which the planner correctly judged cheaper; small differences there are run-to-run noise. Indexes are therefore kept only where a hot query is selective, since each one also slows inserts.

4. Portfolio 3: Transactions and concurrency

- **Commit and rollback:** book_slot() books the slot, records the payment and writes history in one transaction; a failed payment rolls everything back and the slot stays open.
- **Race condition, two applicants and one slot:** the concurrency demos (race_condition_demo.md, run with two live database sessions by run_race_demos.py; real output in race_demos_output.log) show a check-then-insert with no constraint letting both sessions book the same slot, then three fixes: the UNIQUE constraint rejects the second booking; SELECT … FOR UPDATE makes the second session wait and then see the slot closed; SERIALIZABLE aborts the second commit.
- **Isolation and deadlock:** READ COMMITTED shows a non-repeatable read that REPEATABLE READ prevents; two sessions locking two slots in opposite order produce a deadlock, which PostgreSQL resolves by aborting one. Locking in a fixed order prevents it.

5. Portfolio 4: NoSQL (MongoDB)

Uploaded documents (birth certificate, NIN slip, old passport page, photo, affidavit) differ in shape, so they suit MongoDB: each record holds its own type-specific section plus verification results. A $jsonSchema validator (using oneOf) enforces the right section for each document type. With 42,000 documents, a compound index changed the plan from COLLSCAN (42,000 documents examined, 21 ms) to IXSCAN (395 examined, 2 ms). Appointments and payments stay in PostgreSQL, where multi-row transactions and constraints matter.

6. Portfolio 5: Distributed and cloud database

Appointments were range-partitioned by month into 25 partitions plus a default; a one-month query showed "Subplans Removed: 25" (partition pruning). A streaming replica on port 55432 was built with pg_basebackup: the primary reported in_recovery = f and the replica t, an INSERT on the replica was refused ("read-only transaction"), and a row written on the primary appeared on the replica (60,002 → 60,003 on both), with pg_stat_replication showing streaming, async, 0 bytes lag. The cloud design keeps all personal data in Nigeria (NDPA 2023), encrypts it at rest and in transit, limits access by role, and serves the rush when new slots open from read replicas; asynchronous replication favours availability, at the cost of a small data-loss window.

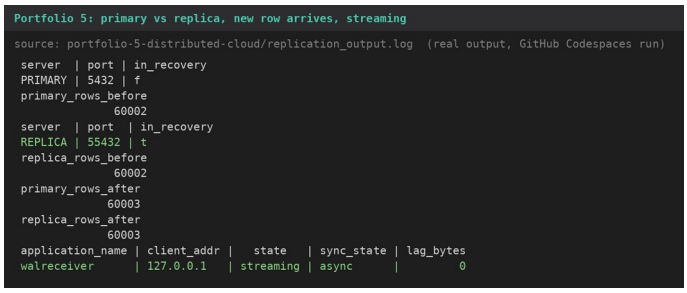


Figure 3. Primary vs replica, the new row arriving, and streaming status (replication_output.log).

7. Individual technical reflection

Working on PassPoint improved my understanding of how advanced database concepts apply to real-world systems. I learned the importance of enforcing data integrity at the database level, optimising queries, managing concurrent transactions and designing for scalability and availability.

Implementing PostgreSQL and MongoDB also helped me understand how to select database technologies based on application requirements. The replication and partitioning exercises further exposed me to practical approaches to improving database performance and reliability.

Although I used AI tools to assist with development, I independently executed, tested and verified the implementations in my GitHub Codespace. Overall, this project strengthened my confidence in designing, implementing and evaluating reliable database systems.

8. AI-use declaration and references

AI tools: Z.ai (GLM) drafted the scripts and documentation; Claude (Anthropic) tested them and fixed the errors found, each listed in CHANGES.md. **My review:** I ran all five portfolios and the six concurrency demos myself in my own GitHub Codespace (the commits and output logs are in the repository), and pulled the key results into one file and checked each against what the brief asks for.

References: Miva Open University, MIT 8103 course notes (2026); Elmasri & Navathe, *Fundamentals of Database Systems* (2016); Kleppmann, *Designing Data-Intensive Applications* (2017); PostgreSQL 16 and MongoDB documentation; Nigeria Data Protection Act 2023. Full list in docs/REFERENCES.md.